OneCompliance

2-Factor Authentication

Specification document

Date: 27-Sept-2021

Version: 1.0

Introduction

This document provides the specification for implementing a multi-factor authentication system within OneCompliance application.

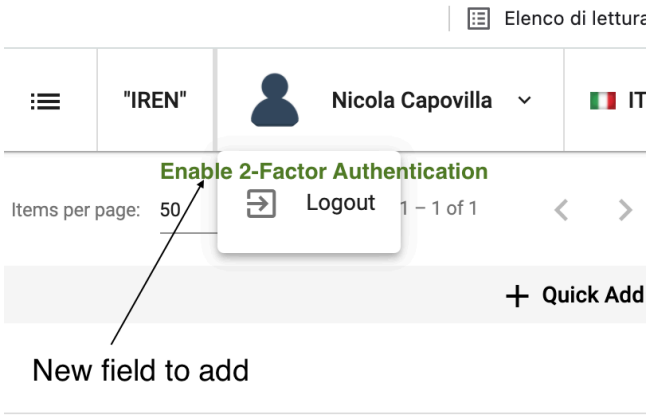
Background

OneCompliance uses the library AWS Amplify (https://docs.amplify.aws) to perform user authentication.
The library can be easily integrated in Angular and provides a convenient abstraction of AWS Cognito service.
Unfortunately we are using a very early and old version (0.1.45) of the library and we cannot move to a newer version as integration with complex lambda function is not supported from version 1.x and above.
So the library documentation might not be fully applicable and usage of APIs should be carefully evaluated

Scenario

Each user can enable/disable the 2FA functionality for her account.
Onecompliance admins can force disabling of the feature on a certain user, however they cannot enable it.
The chosen type of authentication is the so-called TOTP: https://en.wikipedia.org/wiki/Time-based_One-Time_Password
The use case is described by the following steps:

1) The user is already signed in into OneCompliance
2) From the user menu



The user will find an entry which says enable or disable the feature depending on the enabling status.

3) If the feature is already enabled, the user can disable it and a toast message will confirm the deactivation
4) If the feature is not enabled the user can enable it from the menu. On the choice, a pop-up window will present a QR code and the instructions to follow
5) The user must then install a TOTP app, like e.g. “Google Authenticator”, on her smartphone and scan the code
6) A new entry will be presented by the application, providing a 6-digit code that expires (and is renovated) every 30 seconds
7) Pressing a button “Next” on the pop-up, the user is then presented with a request of introducing the 6-digit code for the first time
8) The applications contacts AWS to verify the code, if the check is successful, a proper message is shown and the feature is enables
9) On the next login, once the user introduces her username and password, instead of entering the application a popup requesting the 6-digit verification code is presented.
10) Eventually user logs in only if she enters the correct username and password AND the correct 6-digit verification code from Google Authenticator.

Some technical considerations

Here we can find some details on how to use TOTP with Amplify:

https://docs.amplify.aws/lib/auth/mfa/q/platform/js/#setup-totp

This an example of a procedure to enable the TOTF for a certain user:

```
import { Auth } from 'aws-amplify';
// To setup TOTP, first you need to get a `authorization code` from Amazon Cognito
// `user` is the current Authenticated user
Auth.setupTOTP(user).then((code) => {
    // You can directly display the `code` to the user or convert it to a QR code to be
scanned.
    // E.g., use following code sample to render a QR code with `qrcode.react`
component:
    //      import QRCode from 'qrcode.react';
    //      const str = "otpauth://totp/AWSCognito:"+ username + "?secret=" + code +
"&issuer=" + issuer;
```

```
    //      <QRCode value={str}/>
});

// ...
// Then you will have your TOTP account in your TOTP-generating app (like Google
Authenticator)
// Use the generated one-time password to verify the setup
Auth.verifyTotpToken(user, challengeAnswer).then(() => {
    // don't forget to set TOTP as the preferred MFA method
    Auth.setPreferredMFA(user, 'TOTP');
    // ...
}).catch( e => {
    // Token is not verified
});
```

The procedure represents the steps 2 → 8 of the use case above and must be split across the different views as specified.
So far only the `setupTOTP` API has been tested successfully with version 0.1.45 of Amplify, however this is a good indicator that the full set of MFA related APIs actually works.

setupTOTP() is used to enable the TOTP authentication for the current user and returns a Key (code) that represents a shared secret between the authenticator app on the mobile phone and AWS Cognito service. The key is an important asset to protect and must be used only once to generate the QR code for the user. From there on the frontend application must not store or be provided with the key.

The QR code must represent the following string:

“otpauth://totp/AWSCognito:"+ username + "?secret=" + code + "&issuer=OneCompliance.cloud”

To generate the QR code we can use the library

https://www.npmjs.com/package/ng-qrcode

The state (enabled or disabled) of the feature should be captured as an entry of the user’s JSON on DynamoDB and reported to the frontend to enable (or not) the 2FA flow. This is also the point where the admins can act to force DISABLING (but not enabling) the feature on a certain user.

The user shall not be able to authenticate bypassing the 2FA if enabled.
In order to achieve that, the signIn() function of auth.service.ts must be reviewed (might be already ok) and a new confirmSignIn(code) shall be introduced to verify the 6-digits code.